

Java 8 Functional Interfaces — Cheat Sheet

Quick reference with definitions, method signatures, common patterns, and examples.

Family	Interface(s)	Core method(s)
Predicate	Predicate<T>, BiPredicate<T,U>	test(T), and(), or(), negate()
Function	Function<T,R>, BiFunction<T,U,R>	apply(T), andThen(), compose()
Operators	UnaryOperator<T>, BinaryOperator<T>	apply(T), apply(T,T)
Consumer	Consumer<T>, BiConsumer<T,U>	accept(T), andThen()
Supplier	Supplier<T>	get()
Comparator	Comparator<T>	compare(T,T), comparing(), reverse()

Primitive Specializations

Predicates: IntPredicate, LongPredicate, DoublePredicate

Functions: IntFunction, LongFunction, DoubleFunction, ToIntFunction, ToLongFunction, ToDoubleFunction, IntToLongFunction, IntToDoubleFunction, LongToIntFunction, LongToDoubleFunction, DoubleToIntFunction, DoubleToLongFunction

Operators: IntUnaryOperator, LongUnaryOperator, DoubleUnaryOperator, IntBinaryOperator, LongBinaryOperator, DoubleBinaryOperator

Consumers: IntConsumer, LongConsumer, DoubleConsumer, ObjIntConsumer, ObjLongConsumer, ObjDoubleConsumer

Suppliers: BooleanSupplier, IntSupplier, LongSupplier, DoubleSupplier

Helpful Utilities

Function.identity() — returns a function that returns its input.

Predicate.isEqual(x) — predicate testing Objects.equals(input, x).

BinaryOperator.maxBy/minBy(comparator) — choose max or min by comparator.

Comparator.comparing(keyExtractor), thenComparing() — build comparators with method refs.

Examples

Predicate

```
// Predicate: filter even numbers
List<Integer> nums = Arrays.asList(1,2,3,4,5,6);
Predicate<Integer> isEven = n -> n % 2 == 0;
List<Integer> evens = nums.stream()
    .filter(isEven)
    .toList();
```

Function, compose/andThen

```
// Function: compose & andThen
Function<String, Integer> length = String::length;
Function<Integer, String> describe = n -> "len=" + n;
Function<String, String> pipeline = length.andThen(describe);
String out = pipeline.apply("lambda"); // "len=6"
```

Consumer

```
// Consumer: side effects (e.g., logging)
List<String> names = List.of("Ann", "Bob", "Cat");
Consumer<String> log = s -> System.out.println("-> " + s);
names.forEach(log);
```

Supplier

```
// Supplier: lazy/default values
Supplier<UUID> newId = UUID::randomUUID;
UUID id = newId.get();
```

Operators

```
// UnaryOperator & BinaryOperator: transform & reduce
UnaryOperator<String> trimAndLower = s -> s.trim().toLowerCase();
String res = trimAndLower.apply(" Hello "); // "hello"
```

```
BinaryOperator<Integer> sum = Integer::sum;
int total = Stream.of(1,2,3,4).reduce(0, sum);
```

Comparator helpers

```
// Comparator (functional interface)
record Person(String name, int age) {}
List<Person> people = List.of(new Person("Bob", 30), new Person("Ann", 25));
people.stream()
    .sorted(Comparator.comparing(Person::age).thenComparing(Person::name))
    .forEach(System.out::println);
```

Combined example

```
// Putting it together with Streams
List<String> words = List.of("apple", "banana", "cherry", "date");
Predicate<String> longWord = s -> s.length() > 5;
Function<String, String> upper = String::toUpperCase;
BinaryOperator<String> longest = BinaryOperator.maxBy(Comparator.comparingInt(String::length));

String longestLongWord = words.stream()
    .filter(longWord)
    .map(upper)
    .reduce("", longest);
// Result: "BANANA" or "CHERRY" depending on order
```

Tips & Patterns

- Prefer method references (e.g., `String::length`) for readability.
- Use primitive specializations in tight loops to avoid boxing.
- Chain predicates with `and()`/`or()`/`negate()` rather than nested ifs.
- For reduction, `BinaryOperator.maxBy/minBy` works with a `Comparator`.
- `Function.identity()` is handy in collectors and maps.

Mind Map

